

System.IO.Pipelines: высокопроизводительная передача и обработка потоков данных

C# Developer. Advanced



Проверить, идет ли запись

Меня хорошо видно & слышно?





Антон Герасименко

Lead dev

- Лидирую 2 команды разработки
- Код: dotnet с 2010, typescript, java
- преподаватель курсов
 - все dotnet
 - System design
 - Базы данных
 - System analyst

🔗 @aa_gerasimenko

🔗 @PerpetualMiddle // Вечный миддл



Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе **#канал
группы**



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Маршрут вебинара

Знакомство

Асинхронные методы

Глубокое погружение

Сценарии, Многопоточность

Практика

Основные тезисы

Цели вебинара

К концу занятия вы сможете

1. Использовать `System.IO.Pipelines` для обработки потоков байтов в асинхронных приложениях;
2. Применять пайплайны для эффективного разделения ввода-вывода и логики обработки;
3. Управлять буфером и памятью при работе с большими объёмами данных;
4. Анализировать поведение пайплайнов в многопоточной среде и устранять узкие места



Смысл

Это нужно уметь чтобы:

1. Повышать производительность критически важных систем

2. Уметь работать с большими потоками данных без проблем с памятью

3. Совершенствовать архитектуру и модульность



System.IO.Pipelines

Введение в проблемы высокопроизводительного ввода- вывода в .NET



Проблемы Высокопроизводительного Ввода-Вывода в .NET: До

Проблема 1: Медленные

- Поточковая модель: удобна, но не всегда эффективна для больших объемов.
- Метод часто требует копирования данных между внутренними буферами и буферами пользователя.
- Неэффективное использование памяти: частые аллокации и деаллокации мелких буферов.



Проблемы Высокопроизводительного Ввода-Вывода в .NET: До

Проблема 2: Проблемы с аллокациями

- Частое выделение/освобождение буферов особенно при работе с переменными по размеру пакетами данных.
- Нагрузка на Garbage Collector (GC): частые циклы сборки мусора, "замораживающие" приложение (Stop-The-World), негативно влияют на задержки и пропускную способность.
- Пример: при получении каждого нового сообщения из сети, выделяется новый массив `byte[]`.

```
async Task ProcessLinesAsync(NetworkStream stream)
{
    var buffer = new byte[1024];
    await stream.ReadAsync(buffer, 0, buffer.Length);

    // Process a single line from the buffer
    ProcessLine(buffer);
}
```

Проблемы Высокопроизводительного Ввода-Вывода в .NET: До

Проблема 3: Сложности управления буферами

- Ручное управление: разработчик сам должен следить за выделением, использованием и возвратом буферов.
- Пул буферов помогает, но требует дисциплины и аккуратности.
- "Разорванные" сообщения: необходимо обрабатывать ситуации, когда сообщение приходит не полностью или содержит несколько сообщений.



`System.IO.Pipelines` — это библиотека, предназначенная для упрощения работы с высокопроизводительной операцией ввода-вывода в .NET. Это библиотека, предназначенная для .NET Standard, которая работает во всех реализациях .NET.



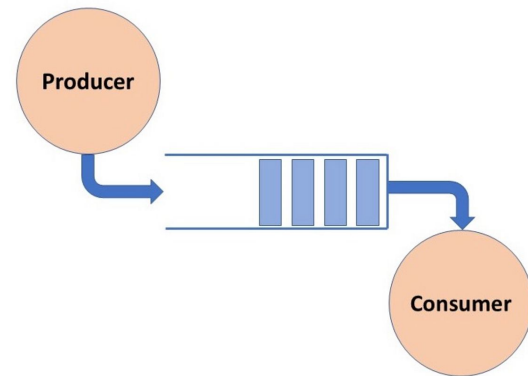
Преимущество 1. Оптимизация аллокаций и избегание копирования

- **Управление памятью:** System.IO.Pipelines использует пулы памяти (например, `MemoryPool<T>`) для эффективного переиспользования буферов.
- **Предоставление прямого доступа к буферам без копирования.**
- **Снижение нагрузки на GC:** Меньше новых объектов, меньше остановок приложения.



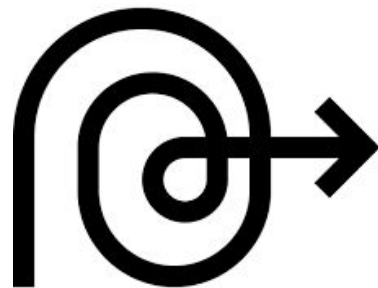
Преимущество 2. Разделение обязанностей (Producer/Consumer)

- **PipeWriter:** Для записи данных (производитель).
- **PipeReader:** Для чтения данных (потребитель).
- **Изоляция логики:** Каждый компонент заботится только о своей части.



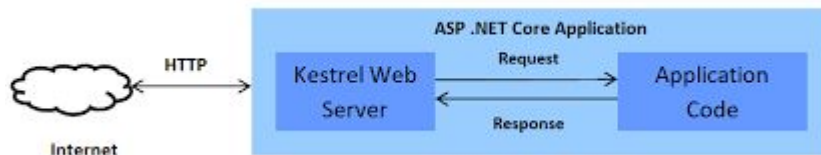
Преимущество 3. Упрощение асинхронного кода

- **Встроенная поддержка `async/await`:**
`ReadAsync/WriteAsync` возвращают `ValueTask`.
- **Обработка `backpressure`:** Автоматическое управление скоростью потока данных между производителем и потребителем.
- **Сложность `TaskCompletionSource` уходит в прошлое:**
Становится проще писать надежный асинхронный код.



Примеры Исползования в Инфраструктуре .NET: Kestrel

- **Сердце ASP.NET Core:** Kestrel – это высокопроизводительный, кроссплатформенный веб-сервер, лежащий в основе всех современных веб-приложений на .NET.
- **Основа эффективности:** System.IO.Pipelines используется Kestrel для чтения HTTP-запросов и записи HTTP-ответов.
- **Решает проблемы:** Позволяет Kestrel обрабатывать миллионы запросов в секунду, минимизируя аллокации и копирования данных.



Пример: gRPC – Высокопроизводительный RPC-фреймворк

- **На базе HTTP/2:** gRPC использует HTTP/2 для эффективной передачи данных.
- **Использование** Обеспечивает высокую пропускную способность и низкую задержку.
- **Эффективная сериализация/десериализация:** Pipelines помогает в передаче сериализованных данных без лишних накладных расходов.

System.IO.Pipelines

1. Значительное сокращение аллокаций и копирований;
2. Упрощение управления буферами;
3. Четкое разделение ответственности;
4. Надежную обработку backpressure;
5. И является основой для таких критически важных компонентов, как Kestrel и gRPC.

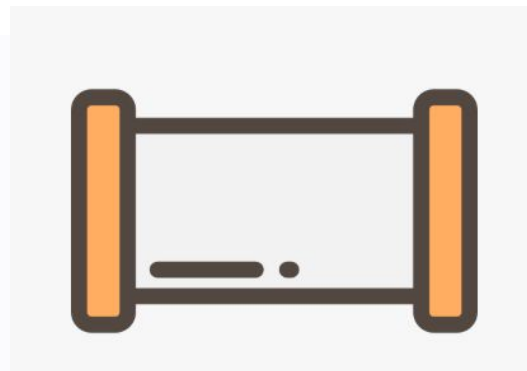


Класс Pipe и его Компоненты



Основная Сущность: Pipe

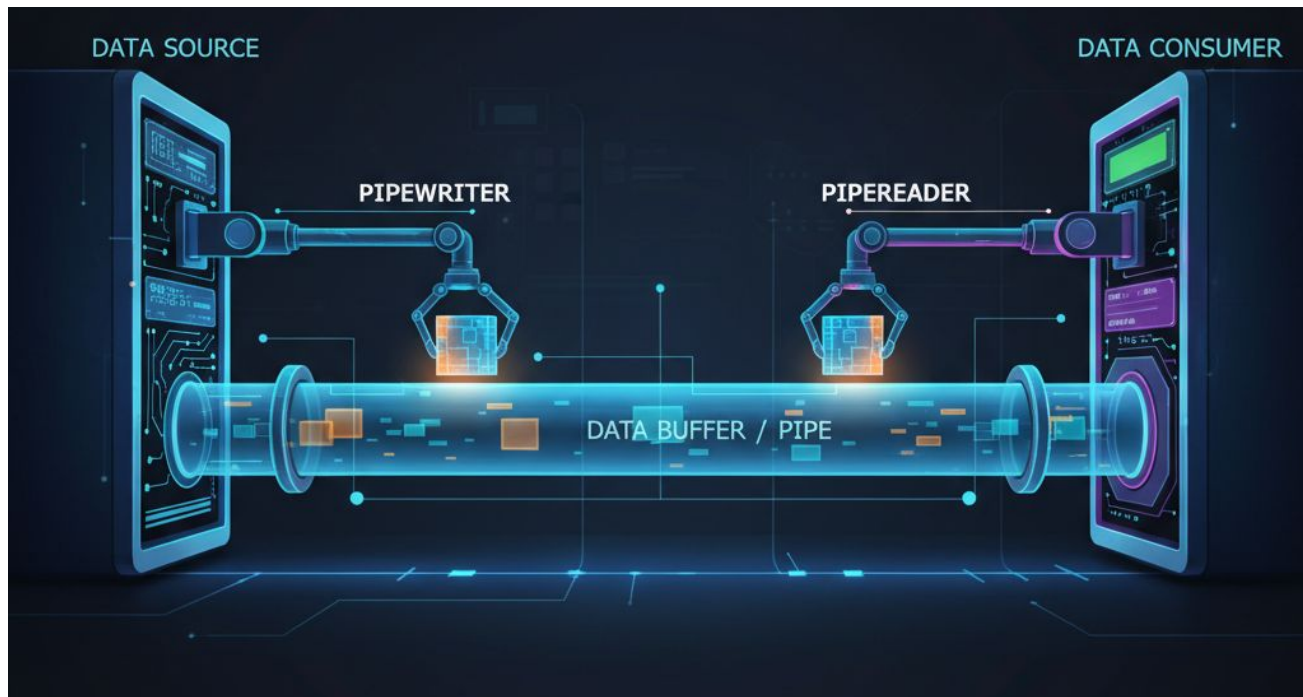
- **Двунаправленный канал:** Pipe – это абстракция двунаправленного канала, который соединяет "производителя" данных с "потребителем".
- **Буферизация:** Внутри Pipe управляет буферами памяти для эффективной передачи данных.
- **Разделение ролей:** Сам Pipe не используется напрямую для чтения или записи. Вместо этого он предоставляет две специализированные конечные точки: `PipeReader` и `PipeWriter`.



PipeReader и PipeWriter. Контракт и Разделение Обязанностей

- PipeWriter: Интерфейс для записи данных в Pipe (сторона "производителя").
 - Предоставляет методы для получения буфера, записи данных и уведомления Pipe, что данные готовы к чтению.
 - Пример: сетевой слушатель, который принимает входящие байты.
- PipeReader: Интерфейс для чтения данных из Pipe (сторона "потребителя").
 - Предоставляет методы для запроса данных, пометки обработанных данных и уведомления Pipe о готовности принять новые данные.
 - Пример: парсер HTTP-заголовков, который читает данные из буфера.
- Преимущество: Четкое разделение ответственности улучшает модульность, тестируемость и предотвращает взаимоблокировки.

PipeReader и PipeWriter. Контракт и Разделение Обязанностей



Инициализация Pipe: new Pipe()

```
using System.IO.Pipelines;

// Создаем новый Pipe с настройками по умолчанию
var pipe = new Pipe();

// Получаем объекты для записи и чтения
PipeWriter writer = pipe.Writer;
PipeReader reader = pipe.Reader;
```


Инициализация Pipe: new Pipe(PipeOptions)

- Кастомизация поведения: Для более сложных сценариев или точной настройки производительности можно использовать конструктор с PipeOptions.
- Настройки PipeOptions:
 - MinimumSegmentSize: Минимальный размер сегмента памяти, который будет выделяться.
 - PauseWriterThreshold: Порог, после которого PipeWriter будет приостановлен, если PipeReader не успевает (Контроль backpressure).
 - ResumeWriterThreshold: Порог, после которого PipeWriter возобновит работу.
 - ReaderScheduler, WriterScheduler: Планировщики для асинхронных операций.
 - UseSynchronizationContext: Использовать ли SynchronizationContext (по умолчанию false для производительности).
 - Pool: Собственный пул памяти (MemoryPool<T>).

ПРИМЕР

Вопросы для проверки

1. Каким образом `System.IO.Pipelines` снижает накладные расходы на выделение памяти по сравнению с чтением данных в `byte[]` из `Stream`?
2. Объясните ключевые компоненты Pipe (`PipeReader` и `PipeWriter`). Какова их основная ответственность и как они взаимодействуют?



Ключевые тезисы

Проблема, которую решает Pipelines:

- Традиционные Stream API часто включают копирование данных между буферами, что вызывает дополнительную нагрузку на CPU и сборщик мусора (GC), особенно при работе с большим количеством мелких операций ввода/вывода.
- Обработка потоков данных вручную с помощью `byte[]` требует сложного управления буферами и их переиспользования, что подвержено ошибкам.
- Синхронный и асинхронный ввод/вывод с помощью Stream API может быть неоптимален для сценариев с высокой пропускной способностью.



Ключевые тезисы

Основные компоненты:

- Pipe: Центральный класс, который представляет собой двунаправленный канал связи. Он состоит из PipeReader (для чтения) и PipeWriter (для записи).
- PipeReader: Позволяет потребителю читать данные из Pipe. Он предоставляет API для получения `ReadOnlySequence<byte>` — высокооптимизированной структуры для представления потенциально несвязанных буферов памяти.
- PipeWriter: Позволяет производителю записывать данные в Pipe. Он предоставляет API для запроса памяти (`Span<byte>`) для записи, а затем подтверждения записи.



PipeWriter



PipeWriter

PipeWriter предоставляет эффективные способы для записи данных, минуя аллокации для каждого записываемого фрагмента. Это достигается за счет использования предварительно выделенных буферов памяти. Ключевые методы, которые мы рассмотрим, позволяют нам получить доступ к этим буферам, уведомить систему о количестве записанных данных и подтвердить их отправку.

Ключевые методы для записи:

- `GetMemory()` / `GetSpan()`: Получение буфера для записи.
- `Advance(int bytesWritten)`: Уведомление о количестве записанных байтов.
- `FlushAsync()`: Подтверждение записи данных в `PipeReader`.
- `Complete()`: Сигнализация об окончании записи.

GetMemory() / GetSpan(): Получение буфера для записи

Методы `GetMemory()` и `GetSpan()` позволяют получить непрерывный блок памяти (`Memory<byte>` или `Span<byte>`), в который мы можем записать наши данные напрямую, без дополнительных копирований.

- `GetMemory(int sizeHint = 0)`: Возвращает `Memory<byte>`. Предпочтительнее, если вы работаете с асинхронными операциями или вам нужен буфер, который может быть использован в течение длительного времени. `sizeHint` - это подсказка для `PipeWriter` о желаемом минимальном размере буфера. Если `PipeWriter` не может предоставить буфер такого размера, он все равно вернет максимально доступный.
- `GetSpan(int sizeHint = 0)`: Возвращает `Span<byte>`.

Advance(int bytesWritten): Уведомление о записанных байтах

- Advance(int bytesWritten): Этот метод принимает один аргумент — bytesWritten, который указывает количество байтов, записанных в буфер, полученный ранее через GetMemory() или GetSpan().

Что происходит после вызова Advance()?

- PipeWriter внутренне обновляет свои указатели, чтобы отразить, что указанное количество байтов теперь готово для чтения PipeReader. Важно вызывать Advance() только после того, как данные действительно записаны в буфер. Если вы записали 100 байтов, а вызвали Advance(50), то только первые 50 байтов будут доступны PipeReader, остальные 50 будут потеряны или перезаписаны в следующих операциях записи.

FlushAsync(): Подтверждение записи данных

- **ValueTask<FlushResult> FlushAsync(CancellationToken cancellationToken = default):** Этот асинхронный метод "проталкивает" записанные данные в PipeReader. Он возвращает ValueTask<FlushResult>, который может быть использован для отслеживания завершения операции.

Что возвращает FlushResult?

- **FlushResult** содержит два важных свойства:
 - IsCanceled: true, если операция записи была отменена CancellationToken.
 - IsCompleted: true, если PipeReader завершил работу (вызвал Complete() или столкнулся с ошибкой). В этом случае дальнейшие записи могут быть бесполезны.

Важность FlushAsync():

- Без вызова FlushAsync(), данные, записанные через GetMemory()/GetSpan() и подтвержденные через Advance(), останутся во внутренних буферах PipeWriter и не будут видны PipeReader. Это критически важный шаг для обеспечения взаимодействия между записывающей и читающей сторонами пайпа.

Complete(): Сигнализация об окончании записи

- **void Complete(Exception? exception = null):** Этот метод сигнализирует PipeReader о том, что PipeWriter закончил свою работу. После вызова Complete() любые последующие попытки записи в PipeWriter приведут к ошибке.

Почему это важно?

- **Завершение чтения:** PipeReader сможет понять, что достигнут конец потока данных и ему не стоит больше ожидать новых данных. Это позволяет PipeReader корректно завершить свою работу.
- **Освобождение ресурсов:** PipeWriter может освободить связанные с ним ресурсы.
- **Обработка ошибок:** Вы можете передать объект Exception в Complete(), чтобы сигнализировать PipeReader о возникшей ошибке во время записи.

Примечание: PipeWriter также имеет асинхронную версию CompleteAsync(), но Complete() является более распространенным.



ПРИМЕР

Вопросы для проверки

1. Опишите типичный цикл записи данных с использованием PipeWriter. Какие методы вы будете использовать для получения буфера, записи в него и уведомления PipeWriter о завершении записи?
2. Что произойдет, если PipeWriter не вызовет Advance() после GetMemory()/GetSpan()? Какие потенциальные проблемы это может вызвать?



**PipeReader и
ReadOnlySequence<byte>**



ReadAsync()

ReadAsync(): возвращает ReadResult

```
public struct ReadResult
{
    public ReadOnlySequence<byte> Buffer { get; }
    public bool IsCanceled { get; }
    public bool IsCompleted { get; }
}
```

`ReadOnlySequence<byte>` – это ключевая структура в экосистеме [System.IO.Pipelines](#).

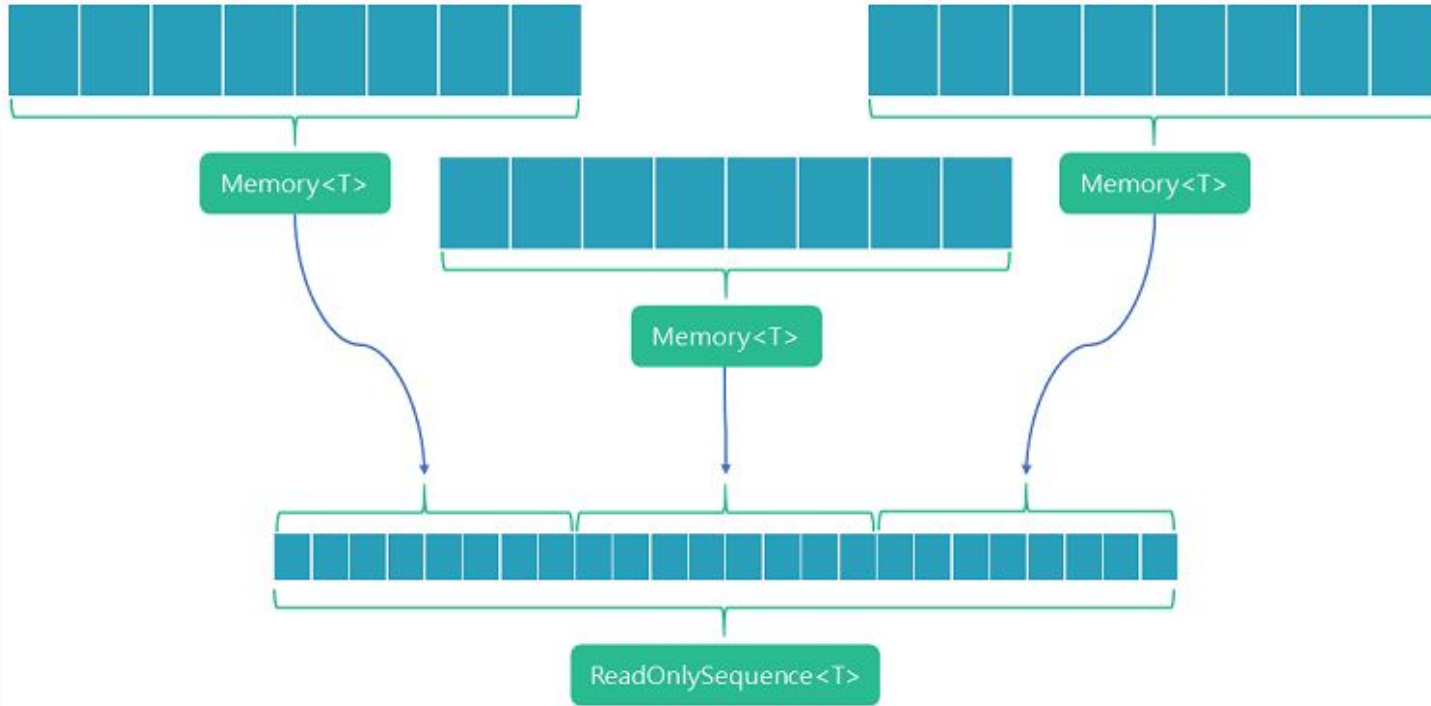
Она представляет собой логически непрерывную, но физически прерывистую последовательность байтов. Проще говоря, это список (или связанный список) блоков памяти (`ReadOnlyMemory<byte>` или `Memory<byte>`), которые вместе образуют единый поток данных.



Почему это важно?

- **Избежание копирований:** Традиционные Stream'ы часто требуют копирования данных в один непрерывный буфер (например, `byte[]`), чтобы вы могли с ними работать. Если данные поступают по частям или хранятся в разных буферах, это приводит к постоянным копированиям. `ReadOnlySequence<byte>` позволяет избежать этого, предоставляя вам "виртуальный" непрерывный буфер, который на самом деле состоит из нескольких "кусков" памяти.
- **Эффективное управление памятью (пулинг):** `System.IO.Pipelines` активно использует пулинг памяти (например, `ArrayPool<byte>`). Когда данные записываются в Pipe, они могут быть разбиты на несколько таких буферов. `ReadOnlySequence<byte>` предоставляет представление этих буферов без их объединения в один. Это позволяет переиспользовать мелкие буферы, уменьшая давление на сборщик мусора (GC).
- **Гибкость:** Она может представлять как один `ReadOnlyMemory<byte>`, так и несколько, что делает ее универсальной для разных сценариев.

ReadOnlySequence<byte>



Как обходить ReadOnlySequence<byte>

Итерация по сегментам (Enumerator)

```
foreach (ReadOnlyMemory<byte> segment in readOnlySequence)
{
    // Работаем с каждым сегментом
    // Span<byte> span = segment.Span;
}
```

GetPosition() и Slice()

- `GetPosition(long offset)`: Возвращает `SequencePosition` по заданному смещению.
- `GetPosition(long offset, SequencePosition origin)`: Возвращает `SequencePosition`, смещенную на `offset` от другой `origin SequencePosition`.
- `Slice(SequencePosition start, SequencePosition end)`: Создает новую `ReadOnlySequence<byte>`, которая является частью исходной, от `start` до `end`.
- `Slice(long start, long length)`: Создает новую `ReadOnlySequence<byte>` длиной `length`, начиная со смещения `start`.

Как Sequence позволяет избежать копирований и аллокаций

Представьте, что вы читаете данные из сети. Данные приходят по частям, и каждый раз `PipeWriter` записывает их в доступный блок памяти из пула.

- Когда вы вызываете `PipeReader.ReadAsync()`, `Pipe` не копирует все эти разрозненные блоки в один большой буфер. Вместо этого он создает `ReadOnlySequence<byte>`, которая ссылается на эти существующие блоки.
- Ваше приложение получает `ReadOnlySequence<byte>` и может обрабатывать данные, перебирая эти сегменты. Нет нужды выделять новый буфер и копировать туда данные.
- После того как вы обработали данные, вы сообщаете `PipeReader` о том, сколько данных было "потреблено" (`AdvanceTo()`). Это позволяет `Pipe` вернуть использованные блоки памяти обратно в пул для повторного использования.

SequenceReader

```
while (!reader.End)
{
    if (reader.TryReadTo(out ReadOnlySpan<byte> item, (byte)','))
    {
        // Обрабатываем найденный элемент (item)
        string itemString = Encoding.UTF8.GetString(item);
        Console.WriteLine(itemString);
    }
    else
    {
        // Разделитель не найден, обрабатываем оставшиеся данные (последний элемент)
        ReadOnlySpan<byte> lastItem = reader.UnreadSpan;
        if (!lastItem.IsEmpty)
        {
            string lastItemString = Encoding.UTF8.GetString(lastItem);
            Console.WriteLine("Last item: " + lastItemString);
        }
        reader.AdvanceToEnd();
    }
}
```

SequenceReader

Недостатки

- Ограниченная область применения (только для `ReadOnlySequence<T>`)
- Нельзя использовать в асинхронных методах (так как это `ref struct`)

ПРИМЕР

Управление Буфером и Памятью



AdvanceTo

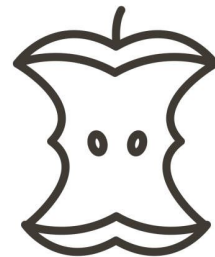
AdvanceTo(): Указание PipeReader, сколько данных было "потреблено". Критически важно для освобождения памяти.

Этот вызов уведомляет PipeReader о том, какая часть данных из ReadOnlySequence<byte> была фактически обработана и, следовательно, может быть возвращена в пул памяти.

```
public void AdvanceTo(SequencePosition consumed);  
public void AdvanceTo(SequencePosition consumed, SequencePosition examined);
```

consumed (Потреблено)

- Это позиция в `ReadOnlySequence<byte>`, до которой вы полностью обработали данные.
- Все данные от начала `ReadOnlySequence<byte>` до `consumed` считаются "съеденными" вашим приложением.
- Память, занимаемая этими "потребленными" данными, может быть возвращена в `ArrayPool` для повторного использования. Это напрямую уменьшает объем активной памяти и снижает нагрузку на сборщик мусора.
- Если `consumed` равна `buffer.End`, это означает, что вы обработали все доступные данные в текущем буфере..



examined (Просмотрено)

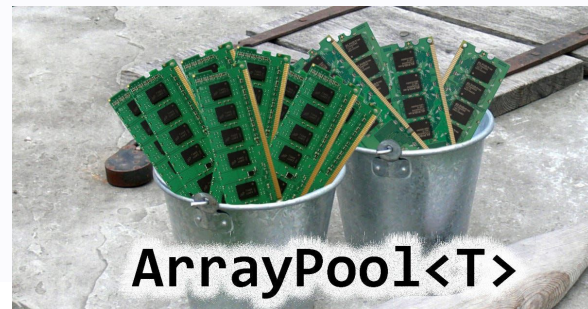
- Это позиция в `ReadOnlySequence<byte>`, до которой вы просмотрели данные.
- Данные между `consumed` и `examined` были просмотрены, но не были полностью обработаны. Например, вы могли просмотреть их в поисках разделителя, но не нашли его.
- Данные до `examined` не будут повторно предоставлены в следующем вызове `ReadAsync()`, пока не будут записаны новые данные после `examined`.
- Это важно для сценариев, когда вы ищете определенный шаблон (например, конец строки) и не находите его в текущем `ReadOnlySequence<byte>`. Вы говорите Pipe, что просмотрели до этой точки, и он должен дождаться новых данных от `PipeWriter`, чтобы продолжить чтение с этой позиции.
- Если `examined` равна `buffer.End`, это означает, что вы просмотрели все данные, которые были доступны в текущем `ReadOnlySequence<byte>`.
PipeReader теперь знает, что может запрашивать больше данных
- Если вы не указали `examined` (используя однопараметрический `AdvanceTo`), по умолчанию `examined` будет равна `consumed`.



Сборка мусора с Pipelines

System.IO.Pipelines спроектированы так, чтобы минимизировать необходимость в сборке мусора.

- Пулинг массивов: Pipe использует `ArrayPool<byte>` для выделения и переиспользования буферов памяти. Вместо постоянного создания новых `byte[]` (что приводит к аллокациям и последующей работе GC), Pipe берет буферы из пула и возвращает их туда, когда они больше не нужны.
- Значимые типы (Structs): `ReadOnlySequence<byte>`, `SequencePosition`, `ReadResult` — все это структуры. Это означает, что они обычно выделяются на стеке, а не в куче, что также снижает нагрузку на GC.



Когда и как вызывается сборка мусора с Pipelines

GC будет вызываться, когда в куче накапливается достаточно объектов, требующих очистки. В контексте Pipelines это может произойти, если:

- Вы не вызываете `AdvanceTo()`. Если вы забыли вызвать `AdvanceTo()`, Pipe не знает, что вы закончили с данными, и не может вернуть буферы в пул. Это приводит к тому, что все больше и больше буферов берутся из пула, а затем становятся непригодными для переиспользования до тех пор, пока Pipe (или `MemoryPool` под ним) не решит их отбросить, что может в конечном итоге привести к GC, если других ссылок на них нет.
- Происходят другие аллокации в вашем коде (например, вы постоянно преобразуете `ReadOnlySequence<byte>` в `string` без оптимизации, или создаете другие объекты). Pipelines помогают с управлением буферами, но не освобождают вас от необходимости оптимизировать другие части кода.

Вопросы для проверки

1. Опишите типичный цикл чтения данных с использованием `PipeReader`. Какие методы вы будете использовать для получения данных, их обработки и уведомления `PipeReader` об их потреблении?
2. Как работает механизм `AdvanceTo()` в `PipeReader`? Объясните разницу между `consumed` и `examined` и почему это различие важно для производительности и корректной обработки данных.



PipeOptions



PipeOptions

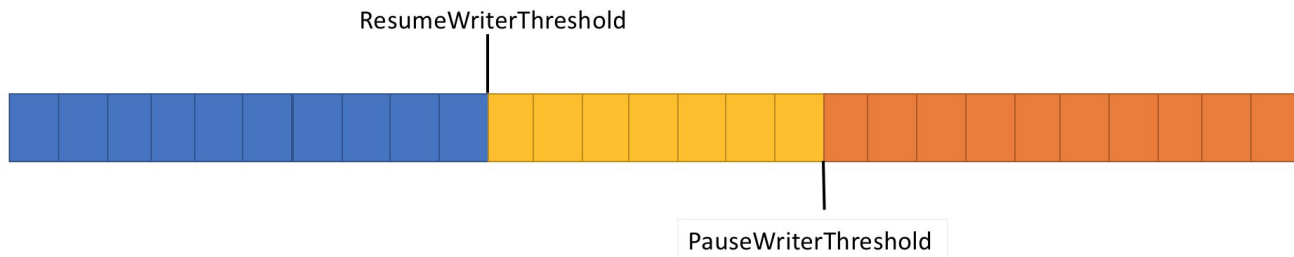
```
public PipeOptions(  
    MemoryPool<byte>? pool = null,  
    PipeScheduler? readerScheduler = null,  
    PipeScheduler? writerScheduler = null,  
    long minimumSegmentSize = -1,  
    long pauseWriterThreshold = -1,  
    long resumeWriterThreshold = -1,  
    bool useSynchronizationContext = true  
)
```

MinimumSegmentSize

- **Влияние:** Определяет минимальный размер сегмента (буфера), который Pipe будет запрашивать из пула памяти. По умолчанию он равен 4096 байтам.
- **На фрагментацию:** Если вы устанавливаете слишком маленький MinimumSegmentSize, это может привести к высокой фрагментации ReadOnlySequence<byte>, то есть она будет состоять из множества мелких сегментов. Это может незначительно замедлить итерацию по ReadOnlySequence и увеличить накладные расходы на управление списками сегментов.
- **На аллокации:** Слишком большой размер может привести к неэффективному использованию памяти, если большинство ваших сообщений очень маленькие. Буфер будет выделен, но большая его часть будет неиспользованной.
- **Оптимальный выбор:** Выбор MinimumSegmentSize зависит от размера типичных данных, которые вы ожидаете. Для сетевых протоколов часто подходят размеры, кратные размеру страницы памяти (4KB) или размеру максимальной единицы передачи (MTU).

PauseWriterThreshold, ResumeWriterThreshold

- Механизмы бэклога (backpressure) для контроля буфера
- **PauseWriterThreshold:** Максимальное количество байтов, которое Pipe может буферизовать между PipeWriter и PipeReader. Когда объем данных в Pipe достигает этого порога, следующий вызов `writer.WriteAsync()` или `writer.FlushAsync()` будет приостановлен до тех пор, пока PipeReader не потребит достаточно данных, чтобы объем буфера опустился ниже `ResumeWriterThreshold`.
- **ResumeWriterThreshold:** Порог, ниже которого объем буферизованных данных должен опуститься, чтобы PipeWriter мог возобновить запись после приостановки. Обычно `ResumeWriterThreshold` меньше `PauseWriterThreshold`.



PauseWriterThreshold, ResumeWriterThreshold

- **Пример:** Если `PauseWriterThreshold = 65536 (64KB)` и `ResumeWriterThreshold = 32768 (32KB)`:
 - PipeWriter записывает данные.
 - Как только в Pipe накапливается 64KB данных, `writer.WriteAsync()` блокируется.
 - PipeReader начинает обрабатывать данные.
 - Когда PipeReader потребляет данные, и объем буфера опускается до 32KB (или ниже), `writer.WriteAsync()` разблокируется, и PipeWriter может продолжить запись.
- **Важность:** Эти пороги предотвращают ситуацию, когда медленный PipeReader приводит к бесконечному накоплению данных в Pipe, что в конечном итоге может исчерпать всю доступную память.

ПРИМЕР

Вопросы для проверки

1. Как `System.IO.Pipelines` обрабатывает обратное давление (backpressure)? Каким образом `Pipe.PauseWriterThreshold` и `Pipe.ResumeWriterThreshold` влияют на это поведение?
2. Объясните концепцию "сегментов" (`ReadOnlySequence<byte>`) в `PipeReader`. Почему данные могут быть представлены не как непрерывный блок памяти, и как это влияет на логику обработки данных?



Обработка ошибок и завершение работы



PipeReader.Complete(), PipeWriter.Complete()

- Методы Complete() являются аналогами Dispose() для PipeReader и PipeWriter. Они сигнализируют о том, что соответствующая сторона Pipe завершила свою работу и больше не будет записывать или читать данные.



PipeWriter.Complete(Exception? exception = null)

- **Что делает:** Сообщает PipeReader, что PipeWriter завершил запись данных. Это устанавливает флаг IsCompleted в ReadResult, который получает PipeReader.
- **Зачем это нужно:** PipeReader может узнать, когда ему больше не нужно ждать новых данных. Если IsCompleted истинно и буфер ReadOnlySequence<byte> пуст, PipeReader знает, что все данные обработаны, и может завершить свой цикл чтения.
- **Обработка ошибок:** Если PipeWriter сталкивается с неустранимой ошибкой, он может вызвать Complete(exception), передав исключение. Это исключение будет выброшено в PipeReader при его следующем вызове ReadAsync(), информируя его об ошибке.
- **Пример:** Сетевое соединение закрылось или возникла ошибка записи в файл.

PipeReader.Complete(Exception? exception = null

- **Что делает:** Сообщает PipeWriter, что PipeReader завершил чтение данных. Это приводит к тому, что будущие вызовы writer.FlushAsync() будут возвращать IsCanceled = true, а также может привести к InvalidOperationException при попытке записать данные.
- **Зачем это нужно:** PipeWriter может прекратить запись, если PipeReader больше не заинтересован в данных (например, клиент отключился).
- **Обработка ошибок:** Если PipeReader сталкивается с неустранимой ошибкой, он может вызвать Complete(exception). Это исключение будет выброшено в PipeWriter при его следующем вызове writer.FlushAsync(), информируя его об ошибке.
- **Пример:** Обработка данных завершилась неудачей или клиентская часть приложения остановилась.

Почему это важно?

- **Избежание зависаний:** Без `Complete()`, одна сторона Pipe может бесконечно ждать данных/потребителя, что приведет к зависанию приложения.
- **Освобождение ресурсов:** `Complete()` помогает Pipe освободить внутренние ресурсы и вернуть буферы в пул. Невызов `Complete()` (или `Dispose()`) может привести к утечкам памяти.
- **Распространение ошибок:** Это стандартный механизм для уведомления другой стороны Pipe о том, что что-то пошло не так.



shutterstock.com · 2580730817

Обработка исключений внутри пайплайна

- **Исключения в PipeWriter:**

- Если PipeWriter сталкивается с исключением (например, ошибка сериализации, потеря соединения), он должен перехватить его и затем вызвать `writer.Complete(exception)`.
- PipeReader при своем следующем вызове `ReadAsync()` получит это исключение.

- **Исключения в PipeReader:**

- Если PipeReader сталкивается с исключением (например, ошибка десериализации, сбой логики обработки), он должен перехватить его и затем вызвать `reader.Complete(exception)`.
- PipeWriter при своем следующем вызове `writer.FlushAsync()` получит это исключение.

- **Неперехваченные исключения:** Если исключение не перехвачено и распространяется вверх по стеку, оно в конечном итоге завершит Task, в котором работает PipeWriter или PipeReader. Это не лучшая практика, так как другая сторона Pipe может зависнуть, не зная о прекращении работы. Всегда старайтесь перехватывать исключения и вызывать `Complete()`!

Вопросы



если есть вопросы



если вопросов нет

Сценарии, Многопоточность и Практика

Сценарии использования

- **Сетевое взаимодействие:** Замена `NetworkStream` на `PipeReader/PipeWriter` для обработки входящих/исходящих сообщений.
- **Потоковая обработка данных:** Чтение из файла, трансформация, запись в другой файл без загрузки всего файла в память.
- **Парсинг протоколов:** Эффективное выделение сообщений из потока байтов (пример: HTTP, WebSocket кадры).
- **IPC (Inter-Process Communication):** Обмен данными между процессами.



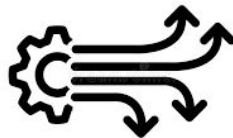
ПРИМЕР

Многопоточность и узкие места

- Pipe по умолчанию потокобезопасен (в определённых пределах).
- PipeReader и PipeWriter не предназначены для одновременного доступа из нескольких потоков (один читатель, один писатель).
- Как использовать Pipelines в многопоточном приложении: отдельные Pipe для каждого соединения/задачи.

Анализ производительности:

- BenchmarkDotNet для измерения производительности.
- Профилирование: на что обращать внимание (аллокации, время ожидания).
- Основные узкие места: слишком частые аллокации (если неправильно использовать AdvanceTo()), ожидание (await).



Вопросы



если есть вопросы



если вопросов нет

**Подведём
итоги занятия**

Ключевые принципы и преимущества:

- Минимизация копирования данных (Zero-copy): Основная цель Pipelines. Вместо копирования данных, он предоставляет ссылки на буферы памяти, где данные уже находятся или будут записаны. Это значительно снижает нагрузку на CPU и уменьшает количество выделений памяти, что уменьшает давление на GC.
- Управление буферами (Buffer Management): Pipelines берет на себя управление пулом буферов (на основе MemoryPool<byte>), автоматически выделяя и освобождая их. Это предотвращает фрагментацию памяти и снижает накладные расходы на GC.
- Обратное давление (Backpressure): Встроенный механизм для управления потоком данных между производителем и потребителем. Если потребитель не успевает обрабатывать данные, PipeWriter автоматически применяет обратное давление, блокируя или приостанавливая производителя, пока потребитель не освободит место. Это предотвращает переполнение буфера и потерю данных.
- Упрощенная асинхронная обработка: Pipelines разработан с учетом асинхронности (ValueTask<T>), что делает его естественным выбором для высокопроизводительных асинхронных операций ввода/вывода.
- Идиоматический API:



Результаты вебинара

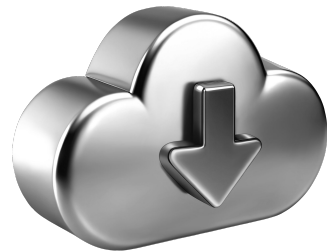
Оцените насколько к концу занятия вы можете:

1. Использовать `System.IO.Pipelines` для обработки потоков байтов в асинхронных приложениях;
2. Применять пайплайны для эффективного разделения ввода-вывода и логики обработки;
3. Управлять буфером и памятью при работе с большими объёмами данных;
4. Анализировать поведение пайплайнов в многопоточной среде и устранять узкие места



Список материалов для изучения

1. <https://learn.microsoft.com/ru-ru/dotnet/standard/io/pipelines>
2. <https://www.stevejgordon.co.uk/an-introduction-to-sequencereader>
3. <https://devblogs.microsoft.com/dotnet/system-io-pipelines-high-performance-io-in-net/>
4. <https://learn.microsoft.com/en-us/dotnet/api/system.buffers.sequencereader-1?view=net-9.0>





Отправьте в чат эмодзи, который отражает ваше настроение после занятия



Продолжите высказывание: теперь я умею/знаю...



Что планируете использовать в работе?

Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу 🖋️❤️